

CSCI 599 Assignment 1 — Tool-Using Agent with MCP Integration

Fall 2026. Individual assignment. 15% of course grade. Due date: see Brightspace.

1. Objective

Build and deploy an LLM-powered agent that reasons over user queries, selects tools exposed via at least two MCP servers, invokes them through the Model Context Protocol, maintains conversational memory across turns, and runs on Google Cloud Run. This assignment demonstrates learning objectives 1, 2, and 4 from the syllabus — agentic architectures and reasoning patterns, production framework implementation with MCP integration, and cloud deployment.

2. Description

You will build one agent — your choice of framework — that connects to at least two MCP servers, reasons about which tool to invoke for a given query, invokes it, incorporates the result into its answer, and remembers the conversation across turns. Your agent runs as a web service on Google Cloud Run and answers queries submitted to a `POST /chat` endpoint (or framework-equivalent).

2.1 Framework choice

Pick one:

- **LangGraph** — recommended path for this course. LangGraph with FastAPI and `langchain-mcp-adapters` is the tested stack. If unsure, pick this. Primary docs: <https://docs.langchain.com/oss/python/langgraph/overview>.
- **OpenAI Agents SDK** — viable path. Use with the Responses API and FastMCP servers. Primary docs: <https://openai.github.io/openai-agents-python/>. MCP integration guidance in the SDK's docs.
- **Google ADK** — viable path. Primary docs: <https://google.github.io/adk-docs/>. MCP integration guidance in the ADK's docs.

Whichever framework you pick, the same rubric applies. Pick the one that matches your prior experience and your appetite for reading unfamiliar framework docs.

2.2 Complexity floor

Your submission must include:

- **≥2 MCP servers integrated.** Canonical recommended pairing: filesystem + Tavily Search (see §3.2). Additional MCP servers beyond two count toward the “Additional MCP servers” rubric criterion (10%).
- **Real MCP protocol invocation.** Your agent must use a real MCP client or framework adapter that performs MCP tool discovery and invocation (`tools/list` / `tools/call`), rather than simulating tool responses. Hard-coded tool responses that never touch the wire are zero-credit on the MCP integration criterion.
- **Conversational memory.** Use your framework’s built-in memory primitives — LangGraph checkpointing, OpenAI Agents SDK sessions, or ADK equivalent. The agent must remember state across at least two turns in the same conversation, keyed by a session identifier passed in the request. For this assignment, conversational memory is required during the lifetime of the running Cloud Run instance; durable persistence across instance recycling is not required.
- **Cloud Run deployment.** A live URL is required for grading. Localhost-only submissions receive zero.

Papa’s rule: An agent that hard-codes tool responses without ever making a real MCP tool invocation is not an MCP integration. Zero credit on the integration criterion regardless of how well the rest of the pipeline works.

3. APIs and LLM Setup

3.1 LLM backend

Recommended order:

1. **OpenAI GPT-5 with API key.** Same pattern as the demo code shown in Weeks 1–3 lectures. API cost should be low at assignment scale; monitor your usage. Set `OPENAI_API_KEY` in your environment; your code reads it from there.
2. **NVIDIA NIM free Developer keys.** Zero-cost alternative. Register at build.nvidia.com to get a free key. Nemotron 3 Ultra and Nemotron 3.5 Lightning are the tested models for this assignment’s task shape. NVIDIA NIM exposes an OpenAI-compatible endpoint, so any framework that accepts a base URL override works — set `OPENAI_BASE_URL` to `https://integrate.api.nvidia.com/v1` and use your NIM key as `OPENAI_API_KEY`. Free-tier rate limit is around 40 requests per minute; adequate for this assignment.
3. **Gemma via Ollama.** Local testing only, never deployment. Useful for local development where you want the agent loop to run without hitting a paid or rate-limited endpoint.

Do not deploy Ollama to Cloud Run. Loading time and memory footprint make it unworkable for a scale-to-zero serverless environment. Your deployed agent must use a cloud-accessible LLM backend.

3.2 MCP servers

You must integrate at least two MCP servers. Canonical recommended pairing:

- **Filesystem MCP** — `@modelcontextprotocol/server-filesystem` (Node). Reads and writes files under a configured root. Zero cost, simplest starting point.
- **Tavily Search MCP** — free tier for search, pay-as-you-go beyond it. Free tier is sufficient for coursework verification runs. Well-supported MCP integration and generous free tier compared to alternatives. Students can receive 4 months of free access to the Project Plan, which includes 4,000 API credits per month.

Additional MCP servers for the 10% “Additional MCP servers” criterion — student choice. Options with confirmed accessible MCP server implementations:

- `@modelcontextprotocol/server-sqlite` (Node). Queries a local SQLite database. Zero cost.
- `mcp-server-fetch` (Python) or the Node fetch equivalent. Fetches URLs and returns page content. Zero cost.
- **Open-Meteo weather MCP** — free weather API, no key required.
- **GitHub MCP server** — requires a free GitHub personal access token. Zero cost within GitHub API rate limits.
- **Student-authored MCP server**. Acceptable and counts toward the requirement. If you build one, expose at least one tool with a valid JSON Schema for the input.

MCP server implementations you use are external open-source projects. Cite them in your README. You do not need to reimplement them.

3.3 MCP protocol surface

MCP uses JSON-RPC 2.0 as its message envelope. Your MCP client or framework adapter must perform at least two MCP operations:

- **tools/list** — discovers the tools exposed by a server, including the JSON Schema for each tool’s input.
- **tools/call** — invokes a named tool with the provided arguments and returns the tool result.

Framework adapters may perform these operations for you. You are not required to construct JSON-RPC envelopes manually.

Your agent must handle three failure modes without crashing:

- **Connection/transport failures.** An MCP server is unavailable or its transport closes during an operation.
- **Tool execution errors.** A valid MCP tool invocation reaches the server, but execution of the requested tool fails. Handle the failure representation returned by the MCP SDK/server.
- **Malformed or invalid responses.** A response cannot be parsed or does not match the expected protocol/schema.

Framework adapters (langchain-mcp-adapters and similar) handle much of the protocol and connection machinery for you. Your application is still responsible for handling failures gracefully rather than crashing or returning an unhandled server error.

3.4 Google Cloud Run

Course-canonical deployment platform. Backed by the \$50 Google Education credit each USC student receives. Cost reference from CSCI 571 last semester: three Cloud Run projects, no student consumed more than \$10 of the \$50 credit. Serverless scale-to-zero keeps idle cost effectively zero.

You will need:

- gcloud CLI installed and authenticated against your USC Google account.
- A GCP project created and billed against the education credit.
- APIs enabled: `run.googleapis.com` and `cloudbuild.googleapis.com`.
- Region `us-west1` recommended (Los Angeles data center, low latency from campus).

4. Cloud Run / Docker Deployment

4.1 Prerequisites

Install and authenticate:

```
# Install gcloud CLI (macOS; see cloud.google.com/sdk/docs/install for other platforms)
brew install --cask google-cloud-sdk
```

```
# Authenticate against your USC Google account
gcloud auth login
```

```
# Set your project
gcloud config set project YOUR_PROJECT_ID
```

```
# Enable required APIs
gcloud services enable run.googleapis.com cloudbuild.googleapis.com
```

4.2 Application shape

Your agent must expose an HTTP endpoint that Cloud Run can serve. The contract:

POST /chat

Request body: { "query": string, "session_id": string }

Response body: { "response": string }

Pseudo-code for the request handler:

```
on POST /chat with body {query, session_id}:
    invoke agent with query
    configure agent memory to use session_id as the checkpoint / thread key
    return { "response": agent_final_message.content }
```

Startup pseudo-code:

```
port ← read PORT environment variable, default 8080
bind server to 0.0.0.0:port
```

Cloud Run injects PORT at runtime. Your code must read it, not hard-code 8080. Frameworks vary in how they wire HTTP endpoints to agent invocation — FastAPI, Flask, and framework-native servers all work. Pick what fits your framework choice.

Papa's rule: A live URL that only responds to your one specific test query fails the deployment side of the code quality criterion. Deploy something that handles any reasonable query, not something that memorized your test cases.

4.3 Dockerfile

Multi-stage build. Non-root user in the final stage. The runtime stage below includes Node.js/npm because the canonical filesystem MCP server is launched through npx:

```
# Build stage
FROM python:3.13-slim AS build
WORKDIR /app
COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

# Runtime stage
FROM python:3.13-slim
WORKDIR /app

# Node.js for @modelcontextprotocol/server-filesystem
RUN apt-get update && apt-get install -y --no-install-recommends nodejs npm &
& \
    rm -rf /var/lib/apt/lists/*

COPY --from=build /root/.local /root/.local
ENV PATH=/root/.local/bin:$PATH
```

```
COPY . .
RUN useradd -m appuser && chown -R appuser:appuser /app
USER appuser
EXPOSE 8080
CMD ["python", "main.py"]
```

Do not use `python:3.13` (unnecessarily large) or `python:3.13-alpine` (musl-libc compatibility issues with common ML dependencies). If your MCP servers are all remote and require no Node runtime, you may omit the Node.js/npm installation.

4.4 Deploy

Copy `.env.example` to `.env`, fill in your own credentials locally, and never commit `.env`. Load those values into your shell before deploying:

```
set -a
source .env
set +a
```

Then build and deploy:

```
# Build and push container image
gcloud builds submit --tag "gcr.io/${GCP_PROJECT_ID}/csci599-a1"

# Deploy to Cloud Run
gcloud run deploy csci599-a1 \
  --image "gcr.io/${GCP_PROJECT_ID}/csci599-a1" \
  --platform managed \
  --region us-west1 \
  --allow-unauthenticated \
  --memory 512Mi \
  --min-instances 0 \
  --max-instances 1 \
  --set-env-vars "OPENAI_API_KEY=${OPENAI_API_KEY},MCP_SERVERS_CONFIG=/app/mcp_config.json"
```

The `gcr.io` image path used here is supported through Artifact Registry's compatible `gcr.io` repository support.

The command prints your live URL:

```
https://csci599-a1-xxxxxxxxxx-uw.a.run.app
```

That's the URL you submit for grading.

`--max-instances 1` keeps the assignment's framework-native in-memory conversational state on a single running Cloud Run instance. This memory is intentionally not durable: it may be lost when Cloud Run stops or replaces the instance. Durable persistence across instance recycling is not required for Assignment 1.

4.5 API keys and environment variables

API keys must never appear in your source repository.

For local development, copy `.env.example` to `.env`, put your local credentials there, load them with `python-dotenv / load_dotenv()` in your Python application, and add `.env` to `.gitignore`. Commit only `.env.example` with placeholder values.

For deployment, source the same local `.env` into your shell and pass the resulting shell variables to Cloud Run through `--set-env-vars`, as shown in §4.4. Do not place literal API keys in the deployment script or repository.

4.6 Required architecture diagrams

Your README must include three diagrams. ASCII, Mermaid, or embedded images are all acceptable. Diagrams that are copy-pasted from framework documentation without adaptation to your actual implementation lose credit.

Diagram 1 — System architecture. Must show:

- FastAPI HTTP endpoint (or framework-equivalent).
- Agent state graph.
- MCP client wrapper.
- Each of your ≥ 2 MCP servers, named individually. Not “MCP Server 1, MCP Server 2” — name the actual servers you integrated (e.g., “filesystem server”, “Tavily search server”).
- LLM backend (OpenAI GPT-5, NVIDIA NIM, whichever you deployed).
- Checkpointer / memory layer.
- Arrows showing request/response flow between all components.

Diagram 2 — Tool invocation flow. Must show the loop:

User query → agent entry → LLM reasoning step → tool selection → MCP tools/call request → MCP server response → LLM synthesis step → final response.

Include the loop-back arrow. When the LLM decides another tool call is needed, it goes back through the reasoning step rather than emitting a final response. Diagrams that show only a single tool call without the loop-back miss the point of agentic reasoning.

Diagram 3 — Deployment topology. Must show:

- Local development environment (Docker + Ollama for local testing, or the same cloud-accessible LLM backend used in deployment).
- Cloud Run deployment path: Docker image → Artifact Registry → Cloud Run service → live URL.
- Environment-variable flow annotated — local `.env` for development and the shell variables passed to Cloud Run through `--set-env-vars`.

5. Submission

Submit four items to Brightspace:

- **Live Cloud Run URL.** Text field. Example format: `https://csci599-a1-xxxxxxxxxx-uw.a.run.app`. Your service must be running and answering queries at grading time. Scale-to-zero keeps idle cost near-zero, so leave it running until grades are posted.
- **Source code ZIP.** Excludes `node_modules`, `virtualenv` directories, `.env`, and anything with API keys. Canonical unprefixed filenames — `README.md`, `main.py`, `deploy.sh`. No `A1_` prefixes. Includes the three required architecture diagrams inside `README.md` or in a linked `docs/` directory.
- **PROCESS_LOG.md.** See §6 for content requirements.
- **README.md** with setup, run, deploy instructions, and the three required architecture diagrams. Bundled inside the source ZIP.

Deadline: per Brightspace.

Late policy: syllabus canonical. Two grace days total across the semester.

6. Academic Integrity

This assignment is subject to USC's academic integrity policies. Specifically:

- **Individual work.** Assignment 1 is an individual assignment. You may discuss general concepts and framework choices with classmates, but the source code, deployment, and written deliverables must be your own work.
- **No sharing of source code** between students. This includes reference implementations (which are instructor-facing and not distributed during the assignment window) and prior semester deliverables.
- **AI-assisted development is allowed and expected.** This is a course about building AI systems. Using LLMs (Claude, GPT, Gemini, Grok, local models) to help write, debug, and refactor code is expected. Using LLMs to substitute for your own understanding of the material is not.
- **Attribution.** Any code copied or substantially adapted from external sources (tutorials, GitHub repositories, framework documentation examples, Stack Overflow) must be attributed in code comments AND in your process log. Attribution format: `# Adapted from <source URL> – <one-line rationale>`. LLM-generated code is a course expectation, not an attribution trigger.

- **MCP servers as external projects.** MCP server implementations you use (filesystem, Tavily, weather, database) are external open-source projects. Attribute their use in the README, but you do not need to reimplement them.
- **Frameworks and libraries.** Using LangGraph, OpenAI Agents SDK, Google ADK, MCP server implementations, Cloud Run deployment tooling, and similar course-canonical tools is expected. These do not require attribution beyond their standard import statements.
- **Sanctions.** Violations are reported to the USC Office of Academic Integrity. Consequences range from assignment failure to course failure to expulsion, per USC policy.

6.1 Process Log

You must submit a Process Log documenting your development process. The process log is worth explicit rubric weight (see §7 — “Code quality and documentation”) and is not optional.

Format: Markdown file named `PROCESS_LOG.md` in the root of your submission. Contents:

1. **AI tools used.** List every LLM you used during development (Claude, ChatGPT, Gemini, Grok, GitHub Copilot, Cursor, local Ollama models, etc.), including which version if you know it.
2. **Development narrative.** Chronological account of how you built the assignment, **200–300 words**. What worked, what didn’t, where you got stuck, how you got unstuck. Include specific AI prompts you used that produced material output.
3. **Verification narrative.** How you tested your code beyond running it once. Did you write tests? Did you inspect tool execution? Did you compare against reference documentation? Did you probe error handling by making an MCP server unavailable? Vibe-coding without verification does not demonstrate mastery.
4. **What you learned.** 2–3 sentences on what the AI tools couldn’t do that you had to figure out yourself. This is the pedagogical anchor — the process log demonstrates the human-in-the-loop learning we teach in this course.

Process log entries that read as if AI wrote them are graded skeptically. Write in your own voice.

7. Rubric

Total: 100% base. No separate bonus.

Criterion	Weight
-----------	--------

Criterion	Weight
MCP integration correctness	25%
Agent reasoning and tool selection	25%
Memory implementation	20%
Code quality and documentation	20%
Additional MCP servers	10%

7.1 MCP integration correctness (25%)

Full credit:

- ≥ 2 MCP servers integrated, each with a configured server manifest reachable at runtime.
- A real MCP client or framework adapter performs tool discovery and invocation (`tools/list` / `tools/call`) against each configured server — not simulated in code.
- Tool invocations produce correct argument shapes matching each tool's declared JSON Schema.
- Error handling covers connection/transport failures, tool execution errors, and malformed or invalid responses. Agent does not crash on any of these.

Partial credit:

- One MCP server integrated cleanly; second server configured but never reached or produces errors.
- Both servers integrated but agent doesn't handle at least one of the three failure modes.
- Tool-call argument shapes wrong for some tools (extra fields, wrong types) but core flow works.

No credit:

- Zero MCP servers actually invoked. Hard-coded tool responses simulating MCP without touching the wire.
- MCP configuration present but agent code doesn't invoke it (dead code).

7.2 Agent reasoning and tool selection (25%)

Full credit:

- Agent selects the appropriate tool for the query — filesystem tool for file-related queries, search tool for research queries, and so on.
- Agent handles queries where no tool is appropriate by answering from LLM knowledge alone rather than forcing a tool call.

- Agent handles multi-step reasoning — chains ≥ 2 tool calls when the query requires it.

Partial credit:

- Agent selects tools but selection is sometimes wrong for the query.
- Agent forces tool calls when it should answer from knowledge alone.
- Agent handles single tool calls but fails to chain multiple tools when the query requires it.

No credit:

- Agent invokes tools at random or based on hard-coded keyword matching that ignores query semantics.

7.3 Memory implementation (20%)

Full credit:

- Conversational memory persists across ≥ 2 turns in the same session while the Cloud Run instance remains running.
- Memory uses the framework's built-in primitives (LangGraph MemorySaver or SQLite checkpoint; OpenAI Agents SDK sessions; ADK equivalent). Custom dict-based implementations lose credit — the point is to demonstrate framework mastery.
- Session identification works. A `session_id` (or thread ID) in the request routes to the correct memory context.
- Agent references prior turn content when the current query depends on it. “What did I ask you last?” produces a real answer, not a generic response.

Durable persistence across Cloud Run instance recycling or scale-to-zero is not required for Assignment 1.

Partial credit:

- Memory persists within a single turn but not across turns.
- Memory works but is not session-scoped (all sessions share state).

No credit:

- No memory implemented. Each request is independent.

7.4 Code quality and documentation (20%)

Full credit:

- `README.md` covers setup, run, deploy, and cost disclosure.
- Architecture diagrams — three required per §4.6 — present, accurate, and match your implementation.

- Source code is readable. Meaningful names, minimal duplication, no dead code, no leftover debug prints.
- `PROCESS_LOG.md` present and substantive per §6.1.
- Live Cloud Run URL responds to arbitrary reasonable queries, not just memorized test cases.

Partial credit:

- README present but missing setup or deploy sections.
- Fewer than three architecture diagrams, or diagrams that don't match your implementation.
- Source has significant dead code or unused imports.
- Process log too short (development narrative <200 words) or reads as AI-generated.

No credit:

- No README, or README is a copy-paste of this assignment description.
- No process log.
- Live URL crashes on any non-trivial query.

7.5 Additional MCP servers (10%)

Full credit:

- ≥ 3 total MCP servers integrated — the 2 required plus at least 1 additional.
- Additional servers reachable through real MCP tool discovery and invocation, using the same interface discipline as the required 2.
- README documents the additional server(s) with rationale for inclusion.

Partial credit:

- Additional server configured but not reached; or reached but no rationale in README.

No credit:

- No servers beyond the required 2.

Regrade policy: per syllabus canonical process. See syllabus for the Piazza form and 2-day window.